STAT 4710J FALL 2026

MIDTERM REVIEW

Guojiao Zhao
zgj_1.17@sjtu.edu.cn

Review Material

Chapter Map

Chapter	Topic	Key Skills
1	Data Fundamentals & EDA	Variable types, primary/foreign keys, sampling frames, data quality, missing values
2	Pandas I & II	Series/DataFrames, indexing, slicing, Boolean filtering, strings, groupby, pivot, merge
3	Text Data: String Methods & Regex	Text normalization, regex syntax, capture groups, Python/Pandas text APIs
4	Text Representation & Embeddings	One-hot encoding, BoW, N-grams, TF-IDF, similarity, Word2Vec, LLM fundamentals
5	Data on the Internet	HTTP、requests、JSON、API、HTML/DOM、BeautifulSoup
6	Visualization	Distribution plots, KDE, boxplots, violin plots, bivariate relationships, visualization design principles
7	SQL I & II	SELECT、WHERE、GROUP BY、HAVING、CASE、CAST、JOIN
8	Machine Learning Supplement	OLS, MSE, cross-validation, regularization, Bias–Variance

Tips

1. The midterm exam will be comprehensive. Make sure you go through all details appeared in slides and labs.
2. Both concepts and code will be tested.
3. The amount of problems might be a lot. Allocate your exam time wisely.
4. For this review file, at least go through all the highlighted part & bold text & in-class problems.

1. Data Fundamentals & Exploratory Data Analysis

1.1 Data Science Lifecycle, EDA, and Data Wrangling

The course describes data science as an iterative lifecycle: Question & Problem Formulation → Data Acquisition → Exploratory Data Analysis (EDA) → Prediction and Inference → Reports, Decisions, and Solutions. In real projects, EDA and Data Wrangling often alternate repeatedly rather than occurring only once.

1.2 Tabular Data: Records, Features, and Variable Types

In tabular data, each row usually represents one observation or record, and each column represents one characteristic or feature. Understanding what each row represents is the first step in determining granularity, keys, and variable types.

Feature Type	Subtype	Meaning	Examples
Quantitative	Continuous	Numeric values can be measured to **arbitrary precision** within a range.	Temperature, price, height, CO_2 concentration
Quantitative	Discrete	Possible values are typically countable, discrete numbers.	Number of siblings, number of orders
Qualitative / Categorical	Ordinal	Categories have a natural order, **but the differences between adjacent categories do not have a consistent numeric meaning**.	Low/medium/high, education level, satisfaction level
Qualitative / Categorical	Nominal	Categories have no natural order.	Color, major, political party, city

In-Class Practice 1A | Classifying Variable Types

A dataset contains rankings of 1, 2, 3, and 4. Which variable type is the best way to interpret these values, and why?

Reference Answers & Explanations

Qualitative ordinal. Rankings have a natural order, but the gaps between ranks do not represent equal numerical differences.

1.3 Primary Keys and Foreign Keys

A primary key is the smallest set of columns that uniquely identifies every record in a table. It may consist of one column or multiple columns (a composite key). A foreign key is a column or set of columns that references the primary key of another table.

Concept	Key Properties	Questions to Check

Concept	Key Properties	Questions to Check
Primary Key	Uniquely identifies each record; distinct records cannot share the same primary-key value.	Are there duplicates? Can one column be removed while uniqueness is preserved?
Composite Primary Key	The combination of multiple columns is unique; an individual column may not be unique by itself.	Is the combination unique? Is there a smaller set of columns that is still unique?
Foreign Key	References the primary key of another table and can be used to join data.	Can each referenced value be found in the referenced table?

```
# Check a candidate key in Pandas
df["record_id"].is_unique


# If an order can contain multiple line items, a composite key may be needed
df[["order_id", "line_number"]].drop_duplicates().shape[0] == len(df)
```

1.4 Population, Sampling Frame, and Sample

Term	Definition
Population	The target population about which we want to learn from the data.
Sampling Frame	The list or frame of units from which sampling can actually be performed.
Sample	The units that are actually selected and observed; a subset of the sampling frame.

The sampling frame does not necessarily match the target population exactly: it may omit some members of the target population or include people who do not belong to it. Therefore, even when a sample is drawn from the frame, it is not necessarily representative of the target population.

2. Pandas I & II: From DataFrames to Data Wrangling

2.1 Series, DataFrames, and Indexes

Pandas is a data-analysis library designed for tabular and heterogeneous data. NumPy is better suited to homogeneous numeric arrays, while Pandas provides labeled Series and DataFrames.

Object	Dimensions	Key Attributes
Series	One-dimensional	One column of values plus a corresponding index; common attributes include .array, .index, and .name.
DataFrame	Two-dimensional	Consists of multiple Series sharing the same index; has row indexes and column labels.
Index	Collection of labels	Can contain numbers or strings; it is not necessarily the default row numbering and is not guaranteed to be unique.

```
import pandas as pd
import numpy as np


s = pd.Series([12, 18, 21], index=["A", "B", "C"])
df = pd.DataFrame({
    "team": ["red", "blue", "green"],
    "points": [12, 18, 21]
})



df.index
df.columns
df.shape      # (number_of_rows, number_of_columns)
df.size       # rows * columns
```

Common Ways to Create a DataFrame

```
# Read a CSV file
df = pd.read_csv("data.csv")
df = pd.read_csv("data.csv", index_col="record_id")
```

```
# Create from a list or nested list
pd.DataFrame([1, 2, 3], columns=["value"])
pd.DataFrame([[1, "A"], [2, "B"]], columns=["id", "group"])

# Create from a dictionary
pd.DataFrame({"fruit": ["pear", "plum"], "price": [4.2, 3.8]})

# Create from Series objects
s.to_frame()
pd.DataFrame({"first": s})
```

Index Operations

```
df = df.set_index("record_id")
df = df.reset_index()


df.index
df.columns
df.shape
```

Common misconception: an Index contains row labels, which are not necessarily the row-position numbers. Index values may also be duplicated. Do not confuse a label with an integer position.

2.2 .loc, .iloc, and [ ]

Syntax	Based On	Slice Endpoint	Typical Use
.loc[row_labels, column_labels]	Label	Includes the right label	Select by row/column labels
.iloc[row_positions, column_positions]	Integer position	*Excludes the right position*	Select by row/column position
df[columns]	Single column name/list of column names	Not applicable	Select one column as a Series or multiple columns as a DataFrame
df[boolean_mask]	Boolean array/Series	Not applicable	Filter rows by condition
df[start:stop]	Common shorthand for row-position slicing	Usually excludes stop	Select consecutive rows

```
# label-based
df.loc[["r1", "r3"], ["city", "score"]]
df.loc["r1":"r3", :]

# integer-position-based
df.iloc[[0, 2], [1, 3]]
df.iloc[0:3, :]

# column selection
df["score"]                 # Series
df[["city", "score"]]       # DataFrame

# first / last n rows
df.head(5)
df.tail(5)
```

Quick reminder: .loc = label; .iloc = integer position. For .loc, both slice endpoints are usually included; for .iloc, the right endpoint is excluded.

In-Class Practice 2A | Labels vs. Positions

1. Given a DataFrame with index labels ['A', 'B', 'C', 'D'], write code to select labels A through C and code to select positions 0 through 2 (excluding position 2).

2. Filter university records with tuition below 50,000. Write both a Boolean-filtering version and a .loc version.

Reference Answers & Explanations

1. Answer: df.loc['A':'C']; df.iloc[0:2]. Note that .loc label slicing usually includes the right endpoint, whereas .iloc positional slicing excludes it.

2. Answer: university[university['Tuition'] < 50000]; university.loc[university['Tuition'] < 50000, :].

2.3 Boolean Filtering: Combining Conditions

First evaluate each condition row by row to produce True/False values, then use the Boolean Series to filter the DataFrame. Combine conditions with & (AND), | (OR), and ~ (NOT). Put parentheses around each comparison expression.

```
# Single condition
df[df["score"] >= 80]


# AND: all conditions must be satisfied
df[(df["score"] >= 80) & (df["city"] == "North")]
```

```
# OR: at least one condition must be satisfied
df[(df["score"] < 60) | (df["status"] == "late")]

# NOT
df[~df["status"].isin(["cancelled", "unknown"])]

# isin: belongs to a set of values
df[df["major"].isin(["Math", "CS"])]

# query: express a condition as a string
df.query('score >= 80 and city == "North"')
```

Incorrect Form	Recommended Form	Reason
df["score"] > 80 and df["city"] == "North"	(df["score"] > 80) & (df["city"] == "North")	Python's and/or cannot be used directly for elementwise Boolean operations on Series.
df["score"] > 80 & df["city"] == "North"	(df["score"] > 80) & (df["city"] == "North")	Parenthesize comparison expressions to avoid operator-precedence errors.
df.iloc["score", :]	df.loc[:, "score"]	.iloc accepts integer positions, not column names.

In-Class Practice 2B | Boolean Masks

1. Keep records where score is at least 80 and campus is 'East'. Write the correct Pandas expression.

2. The Accepted column stores the strings '0' and '1'. When filtering accepted records, should you compare against 1 or '1'?

3. Determine whether all records with open_face=1 also have lattice=0. Write a Boolean expression.

Reference Answers & Explanations

1. Answer: df[(df['score'] >= 80) & (df['campus'] == 'East')]. Put parentheses around each comparison and use &.

2. Answer: Compare against the string '1': df[df['Accepted'] == '1']. The stored data type determines the comparison value. Notice that Accepted column stores str instead of int. **Don't forget quotes**!

3. Answer: (df.loc[df['open_face'] == 1, 'lattice'].sum() == 0).

2.4 String Methods: Pandas String Columns

```
# Normalize text
df["city"] = df["city"].str.strip().str.lower()
```

```
# Find rows containing specified text
df[df["description"].str.contains("urgent", na=False)]


# Check whether a string starts with a specified prefix
df[df["code"].str.startswith("AB")]


# Split a string and select one component
df["state"] = df["location"].str.split(", ").str[1]
```

```
# String length
df["name_length"] = df["name"].str.len()


# String slicing
df["prefix"] = df["code"].str[:3]
```

Pandas String Method	Purpose
.str.lower() / .str.upper()	Normalize letter case
.str.strip()	Remove leading/trailing whitespace
.str.replace(old, new)	Replace text; check the regex argument/version behavior when using regex
.str.split(sep)	Split by a separator; use .str[index] to select a component
.str.contains(pattern)	Test for text/regex pattern and return a Boolean Series
.str.startswith(prefix)	Check whether the string starts with a specified prefix
.str.len()	Calculate string length
.str.findall(pattern)	Return a list of all matches for each row
.str.extract(pattern)	Extract capture groups from the first match
.str.extractall(pattern)	Extract capture groups from all matches

In-Class Practice 2C | Extracting Strings

1. The Location column has the format 'City, State'. Create a State column containing only the state name.

2. In the soda column, select records that contain a space and have a string length of at least 4.

Reference Answers & Explanations

1. Answer: df['State'] = df['Location'].str.split(', ').str[1].

2. Answer: has_space = df['soda'].str.contains(' ', na=False); result = df[has_space & (df['soda'].str.len() >= 4)].

2.5 Adding, Modifying, Renaming, and Dropping Columns

```
# Add a new column

df["score_plus_5"] = df["score"] + 5



# Modify an existing column

df["score"] = df["score"].clip(0, 100)



# Rename columns

df = df.rename(columns={"score_plus_5": "adjusted_score"})



# Drop columns

df = df.drop("adjusted_score", axis="columns")



# Drop rows by index label

df = df.drop("r2")
```

Copy semantics: The course emphasizes that most Pandas methods return a new object by default and do not automatically modify the original DataFrame. Reassign the result to a variable (for example, df = df.drop(...)).

2.6 Handy Utility Functions

Function / Attribute	Purpose	What to Watch For
.shape	Returns (rows, columns).	It is a tuple.
.size	Returns rows × columns.	It is not the row count.
.describe()	Descriptive-statistics summary.	The summary may differ for a Series and a DataFrame.
.sample(n, replace=False)	Random sampling.	Sampling is without replacement by default; use replace=True for

Function / Attribute	Purpose	What to Watch For
		sampling with replacement.
.value_counts()	Count the frequency of each unique value in a Series.	Returns a Series, usually sorted by frequency in descending order.
.unique()	Returns an array of unique values.	Not equivalent to value_counts.
.sort_values()	Sort by values.	For a DataFrame, specify by; ascending order is the default.
.map(func)	Apply a function to each element of a Series.	Useful for elementwise mapping or custom sort keys.
.apply(func)	Apply a function to Series elements or along a DataFrame axis.	Understand whether the input is a single element or an entire row/column.

```
df.describe()

df["category"].value_counts()

df["category"].unique()

df.sample(n=5, replace=False)

df.sort_values(by="score", ascending=False)


df['name_upper'] = df['name'].map(str.upper)


def row_average(row):
    return (row['score1'] + row['score2']) / 2
df['avg'] = df.apply(row_average, axis=1)

print(df[['name', 'score1', 'score2', 'avg']])

#       name  score1  score2   avg

# 0    Alice      85      90  87.5

# 1      Bob      92      88  90.0

# 2  Charlie      78      85  81.5

# 3    Diana      88      95  91.5



# Sort by string length
```

```
df.sort_values("name", key=lambda s: s.str.len(), ascending=False)
```

In-Class Practice 2D | value_counts and Plotting

Given fav = df['flavor'].value_counts(), how should you pass the category labels and counts to Matplotlib to draw a bar chart of category frequencies?

Reference Answers & Explanations

Answer: plt.bar(fav.index, fav.values). You can also use seaborn's sns.countplot(data=df, x='flavor'). Do not interpret a category-frequency Series as a histogram of a continuous numeric variable. See the notebook file.

2.7 GroupBy: Split → Apply → Combine

groupby splits data into sub-DataFrames based on one or more keys, applies an operation to each group, and combines the results. Calling groupby() returns a GroupBy object, not a regular DataFrame. See notebook.

```
# Group by one column and aggregate

df.groupby("department")["score"].mean()


# Group by multiple columns

df.groupby(["year", "department"])["score"].mean()


# agg: summarize each group with one or more statistics

df.groupby("department").agg(np.mean)

df.groupby("department").agg("max")


# Common aggregation methods can be called directly

df.groupby("department").sum()

df.groupby("department").first()

df.groupby("department").last()
```

Differences Among Common GroupBy Operations

Method	Return Value / Behavior	Suitable Questions
.agg(func)	Compress each group into summary value(s)/row(s).	Group means, maxima, sums, and similar summaries.
.filter(func)	func receives each group's DataFrame and returns True/False; if True, all original rows in that group are retained.	Retain groups whose mean/total meets a condition.
.size()	Number of rows in each group, including rows where a field is NaN.	Count the number of records in each group.

Method	Return Value / Behavior	Suitable Questions
.count()	Return the number of non-missing values in each column.	Count valid values in each column for each group.
.get_group(key)	Retrieve a specific group.	Inspect the original records in a particular group.

```
# Keep departments with an average score above 75; retain all rows from those departments
df.groupby("department").filter(
    lambda group: group["score"].mean() > 75
)

# Number of records in each department
df.groupby("department").size()


# Number of non-missing score values in each department
df.groupby("department")["score"].count()
```

size vs. count: size() counts rows in each group; count() counts non-missing values in each column. The results may differ when NaN values are present.

In-Class Practice 2E | GroupBy, filter, and agg

1. Keep store groups whose total sales exceed 1,000, retaining all original rows from those groups.
2. Group by product_type and region, calculate the mean rating, and sort the results from highest to lowest mean.
3. What is the difference between groupby(...).size() and groupby(...).count()?

Reference Answers & Explanations

1. df.groupby('store').filter(lambda g: g['sales'].sum() > 1000). filter retains all original rows from groups that satisfy the condition.

2. df.groupby(['product_type','region'])['rating'].mean().sort_values(ascending=False) 。

3. size() counts rows in each group; count() counts non-missing values in each column. They may differ when NaN values are present.

2.8 Pivot Tables and GroupBy

```
summary = df.pivot_table(
    index="year",         # output rows
    columns="department", # output columns
    values="score",       # field to aggregate
```

```
    aggfunc=np.mean       # aggregation function
)
```

pivot_table is useful when one grouping dimension is placed in rows and another in columns, creating an intuitive cross-tabulation. groupby(['year', 'department']) often produces a MultiIndex.

2.9 Joining Tables: pd.merge [Really important; See notebook]

```
merged = pd.merge(
    left=orders,
    right=customers,
    how="inner",
    left_on="customer_id",
    right_on="id"
)
```

Join Type	What Is Retained	Unmatched Rows
inner	Rows with matching keys in both tables.	**Unmatched rows are dropped.**
left	All rows from the left table plus matching rows from the right table.	**Unmatched right-table fields are filled with NaN.**
right	All rows from the right table plus matching rows from the left table.	**Unmatched left-table fields are filled with NaN.**
outer	All rows from both tables.	**Fields on the missing side are filled with NaN.**

- **on: use this when the key column has the same name in both tables; left_on / right_on: use these when the key columns have different names.**
- left_index / right_index: specify whether to use the index as the join key.
- **First check the granularity and uniqueness of the key: one-to-many, many-to-one, and many-to-many merges produce different row counts. See notebook.**
- **After merging, check the row count, duplicates, missing values, and column names to confirm that the result is as expected.**

In-Class Practice 2F | Merge + Sort + GroupBy

The join key in orders is customer_id, while the join key in customers is id. First perform an inner merge. Then assume the merged result contains State and Ranking; find the highest-ranked school in each state (the smallest Ranking value).

Reference Answers & Explanations

Answer: merged = pd.merge(orders, customers, how='inner', left_on='customer_id', right_on='id').

Answer: best = complete.sort_values('Ranking').groupby('State').first(). Sort by ranking in ascending order, then group by state and take the first row.

2.10 Common Pandas Errors to Check

Symptom	Common Cause	How to Fix
Incorrect filtering result / error	Missing parentheses in multiple conditions, or incorrect use of and/or.	Parenthesize each comparison and use & / | / ~.
KeyError	Column name or index label does not exist; capitalization/spacing differs.	Check df.columns and df.index.
iloc raises an error	A column name was passed to iloc.	Use iloc for positions and loc for labels.
The row count increases dramatically after merge	The join key is not unique on one or both sides, creating a one-to-many or many-to-many join.	Check key uniqueness and relationship cardinality first.
The original DataFrame is unchanged after drop/rename	The result was not reassigned.	df = df.rename(...) / df = df.drop(...)。
The result has far fewer rows after groupby	agg compresses each group into a summary.	If you need to retain original rows within each group, consider filter.

3. Text Data： String Methods & Regular Expressions

3.1 Why Process Text? Two Main Goals

Goal	Meaning	Examples
Canonicalization	Convert different textual representations of the same information into a standard form.	Standardize capitalization, remove spaces, standardize abbreviations, and remove punctuation to support joins/grouping.
Extraction	Extract information from text to create new features.	Extract dates, times, IP addresses, and IDs from log lines.

```
def canonicalize_label(s):
    return (
        s.lower()
         .replace(" ", "")
         .replace("&", "and")
         .replace(".", "")
    )
```

3.2 Python Strings and the Pandas .str API

Operation	Python string	Pandas Series
lower / upper	s.lower() / s.upper()	ser.str.lower() / ser.str.upper()
replace	s.replace(old, new)	ser.str.replace(old, new)
split	s.split(sep)	ser.str.split(sep)
slice	s[1:4]	ser.str[1:4]
contains	'ab' in s	ser.str.contains(pattern)
length	len(s)	ser.str.len()
strip	s.strip()	ser.str.strip()
regex replace	re.sub(pattern, repl, s)	ser.str.replace(pattern, repl, regex=True)
find all	re.findall(pattern, s)	ser.str.findall(pattern)

3.3 Basic Regex Syntax

A regular expression (regex) is a sequence of characters that describes a search pattern. The course emphasizes metacharacters, closure, escape characters, character classes, and groups.

Syntax	Meaning	Example / Explanation
AB	Concatenation: B immediately follows A.	r'cat' matches the consecutive sequence cat.
A|B	OR: match A or B.	r'cat|dog'。
(AB)	Group: treat a pattern as a unit; also creates a capture group.	r'(ab)*'。
A*	The preceding element appears zero or more times.	r'bo*k'。
A+	The preceding element appears one or more times.	r'bo+k'。
A?	The preceding element appears zero or one time.	r'colou?r'。
A{m}	The preceding element appears exactly m times.	r'\d{4}'。
A{m,n}	The preceding element appears from m to n times.	r'\w{2,5}'。
.	Any single character (usually excluding a newline).	r'a.b'。
[abc]	Match any one character inside the square brackets.	r'[aeiou]'。
[a-z]	A character range.	r'[A-Za-z]'。
[^abc]	Match one character not in the set; ^ at the beginning of a character class negates the set.	r'[^0-9]'。
\d / \w / \s	Digit / word character / whitespace character.	r'\d+'、r'\w+'、r'\s+'。
\D / \W / \S	The corresponding non-digit / non-word / non-whitespace characters.	r'\D+'。
\.	Escape: match a special character literally.	Match a literal period rather than any character.
^ / $	Anchors for the beginning and end of a string/line.	r'^A'、r'end$'。

Most important distinction: () defines groups; [] defines character classes; {} specifies repetition counts. These have completely different purposes.

In-Class Practice 3A | Regex Patterns

. The professors store a `RegEx` pattern in the variable `soda_pattern` which extracts the following substrings in the example below:

```
re.findall(soda\_pattern, "Some people like Diet Coke or Cherry Coke more than regular Coca-Cola")

Output: ["Coke", "Coke", "Coca-Cola"]
```

Which of the following `RegEx` patterns could be `soda_pattern`? You only need to worry about the example text above and do not need to consider cases that are not present in the example. Select all that apply.

(a) `r"Co[ke|ca\-Cola]"`

(b) ✔ `r"Co\w{2}\-? \w*"`

(c) ✔ `r"Coke|Coca-Cola"`

(d) `r"Co(ke|ca-Cola)"`

See notebook.

3.4 Regex Operator Precedence and Parentheses

In regex, concatenation has higher precedence than OR (|). If OR should apply only to a local part of the pattern, use parentheses to make its scope explicit.

```
# Match a pattern that starts with m and ends with n, with an even number of at least two u's or o's in between

r"m(uu(uu)*|oo(oo)*)n"



# Incorrect grouping changes the scope of OR

# r"m(uu(uu)*)|(oo(oo)*)n"  # This represents two complete alternatives
```

3.5 Raw Strings: Why r'...' Is Recommended

Both Python strings and regex use the backslash as an escape character. Patterns are usually written as raw strings (r'...' or r"..."), which reduces confusion caused by double escaping.

```
pattern = r"\d{3}–\d{2}–\d{4}"

text = "reference: 482–17–6390"

re.findall(pattern, text)
```

3.6 Python re and Pandas Regex Functions

Function	Functionality	Return Value
re.sub(pattern, repl, text)	Replace all matches.	New string

Function	Functionality	Return Value
re.findall(pattern, text)	Find all matches.	A list; **if the pattern contains capture groups, returns group contents/tuples.**
ser.str.findall(pattern)	Find all matches in each row of a Series.	Series of lists
ser.str.contains(pattern)	Check whether a match is present.	Boolean Series
ser.str.extract(pattern)	Extract capture groups from the first match.	DataFrame; each capture group corresponds to one column.
ser.str.extractall(pattern)	Extract capture groups from all matches.	DataFrame; one row per match.

Capture Groups

Each pair of ordinary parentheses defines a capture group. **If a re.findall pattern contains multiple capture groups, the result is usually a list of tuples.**

```
text = "Log 07:18:42 – device restarted"

pattern = r"(\d{2}):(\d{2}):(\d{2}) – (.*)"



re.findall(pattern, text)

# [('07', '18', '42', 'device restarted')]



# Pandas: extract multiple components into columns

ser.str.extract(r"(\d{2}):(\d{2}):(\d{2})")
```

In-Class Practice 3B | findall and Capture Groups

When r'([A-Z]{2})-(\d{4})' is applied to the text 'ID=AB-4821; ID=CD-9300', what structure does re.findall return?

How should the pattern be rewritten if you want to extract each complete ID rather than the two separate groups?

Reference Answers & Explanations

Answer: [('AB', '4821'), ('CD', '9300')]. The pattern has two capture groups, so findall returns a list of tuples.

Answer: Use r'[A-Z]{2}-\d{4}'. Without capture groups, findall returns a list of complete matched strings.

See notebook

3.7 Worked Patterns

Task	**Pattern**	Explanation
Extract a four-digit year	r'\b\d{4}\b'	Four digits; \b represents a word

Task	Pattern	Explanation
		boundary (boundary syntax is not a primary focus of this material).
Extract a word beginning with an uppercase letter	r'[A-Z][a-z]+'	One uppercase letter followed by one or more lowercase letters.
Extract two letters + hyphen + four digits	r'([A-Z]{2})-(\d{4})'	The two capture groups extract the prefix and the number separately.
Match the emoji-name format :name:	r':([^:]*):'	Matches non-colon characters between colons; the capture group does not include the colons.
Remove HTML tags (simple example)	r'<[^>]+>'	A < followed by one or more non-> characters and then a >.

3.8 Common Errors

- Treating . as a literal period: in regex, . usually means any single character; a literal period must be written as \.
- Confusing [] with (): [] defines a character class, not a capture group.
- Forgetting raw strings, especially for patterns such as \d, \w, and \s.
- Assuming findall always returns complete matches: when capture groups are present, it may return group values or tuples instead.
- Using regex to parse JSON or complex nested structures: the course explicitly states that hierarchical structures should be handled with the appropriate parser, not regex.
- Regex is not suitable for tasks such as fully validating complex email addresses or counting nested parentheses.

4. Text Representation & Embeddings

4.1 Tokenization and Vocabulary

Computers cannot directly use natural language as numeric features. Text is usually split into tokens first, and a vocabulary is then built. English tokenization often uses spaces as a guide; in languages such as Chinese, word boundaries are not necessarily marked by spaces, so a tokenization algorithm is needed.

4.2 One-Hot Encoding

If a categorical variable has k possible values, create one new dimension for each category. Each record has a 1 in the dimension corresponding to its category and 0s in all other dimensions.

Category	Encoding Illustration (Vocabulary = [oak, pine, maple])
oak	(1, 0, 0)
pine	(0, 1, 0)
maple	(0, 0, 1)

- The vector length equals the vocabulary size.
- Each category corresponds to a separate dimension.
- One-hot encoding is suitable for representing category identity, but it does not naturally express semantic similarity or ordering between categories.

4.3 Bag-of-Words (BoW)

BoW extends one-hot representation to an entire text: each vector dimension corresponds to a word, and the value records how many times that word appears in the document. A bag is a multiset: order does not matter, and repeated elements are allowed.

```
Vocabulary = ["data", "science", "is", "useful"]



Document: "data science is useful and data"

BoW (using the vocabulary above) = [2, 1, 1, 1]
```

Key Limitations of BoW

Issue	Explanation
High Dimensionality	A real vocabulary may contain a large number of words, resulting in high-dimensional vectors.
Sparsity	A single document contains only a small subset of the vocabulary, so most vector positions are 0.
Loss of Word Order	Word order is lost; sentences with the same word counts may produce identical BoW vectors.
Out-of-Vocabulary (OOV)	Words outside the vocabulary may be ignored or mapped to UNK, causing information loss.

In-Class Practice 4A | BoW Vocabulary

Given three documents: 'data helps teams', 'teams need context', and 'data needs context'. Without removing stop words, how many columns will the BoW matrix have? Does it preserve word order?

Reference Answers & Explanations

Answer: The vocabulary is data, helps, teams, need, context, and needs: six distinct tokens, so the matrix has six columns. BoW preserves word counts but not word order.

4.4 N-Gram Encoding

An N-gram treats each sequence of N consecutive words as a feature. For example, a 2-gram (bigram) consists of adjacent word pairs. It preserves some local word-order information, but the number of possible combinations increases, the vectors become sparser, and prediction may encounter combinations not seen during training.

Sentence	2-gram Example
the book was well written	the book; book was; was well; well written
we study data science	we study; study data; data science

4.5 TF, IDF, and TF-IDF

TF-IDF aims to measure how representative a word is of a particular document. The more frequently a word appears in that document (higher TF), the more relevant it may be; however, if it is common across all documents (lower IDF), it has less power to distinguish that document.

Quantity	Formula (N is the total number of documents)	Intuition
Term Frequency	tf(t,d) = number of occurrences of term t in document d / total number of words in document d	Measures the relative frequency of a term in the current document.
Document Frequency	df(t) = number of documents containing term t	Counts the number of documents in which the term appears, not its total number of occurrences.
Inverse Document Frequency	idf(t) = log(N / df(t))	The rarer a term is, the larger its IDF; the more common it is, the smaller its IDF.
TF-IDF	tfidf(t,d) = tf(t,d) × idf(t)	Terms that are common in the current document but rare in the overall corpus receive higher weights.

Calculation note: use the logarithm base specified in the question; if base-2 is specified, use $\log_2$. The denominator of TF is the total number of words in the current document; the denominator used in IDF is the number of documents containing the term.

Worked Calculation

The corpus contains three documents: D1 = “red fox red”; D2 = “blue fox”; D3 = “red bird”. Use a base-2 logarithm.

Term and Document	TF	DF	IDF	TF-IDF
red in D1	2/3	2	$\log_2(3/2) \approx 0.585$	≈ 0.390
blue in D2	1/2	1	$\log_2(3) \approx 1.585$	≈ 0.793
fox in D1	1/3	2	$\log_2(3/2) \approx 0.585$	≈ 0.195

In-Class Practice 4B | TF-IDF by Hand

A corpus contains four documents. The word 'orbit' appears in two of them. In document D1, 'orbit' appears three times, and D1 contains 12 words in total. Using $\log_2$, calculate the TF, IDF, and TF-IDF of 'orbit' in D1.

Reference Answers & Explanations

Answer: TF = 3/12 = 0.25; DF = 2 and N = 4; IDF = $\log_2(4/2) = 1$; TF-IDF = $0.25 \times 1 = 0.25$.

4.6 Similarity: Dot Product and Cosine Similarity

The key intuition from the course is that the more similar two unit vectors are, the larger their dot product. Cosine similarity measures directional similarity using the angle between vectors, avoiding a similarity measure determined only by vector length.

$$\cos(u, v) = (u \cdot v) / (\|u\| \|v\|)$$

```
# Conceptual calculation

dot = sum(a * b for a, b in zip(u, v))

norm_u = (sum(a*a for a in u)) ** 0.5

norm_v = (sum(b*b for b in v)) ** 0.5

cosine = dot / (norm_u * norm_v)
```

4.7 Word2Vec：Context-Based Embedding

Word2Vec uses neural networks to learn word associations from large text corpora. Word vectors are learned from the contexts in which words appear; words that occur in similar contexts are usually closer together in vector space.

Method	Input	Prediction Target
CBOW (Continuous Bag-of-Words)	Context words	Center word
Skip-Gram	Center word	Surrounding context words

Compared with BoW, Word2Vec is a distributed representation learned from context, whereas BoW/TF-IDF are count-based representations.

4.8 Contextual Embeddings and LLM Concepts

- Lecture 05 introduces the progression from static word embeddings and neural language models to attention/Transformers, contextual word embeddings, and prompting.

- GPT stands for Generative, Pretrained, and Transformer.
- Models typically process tokens rather than necessarily complete words. A token may be a word, part of a word, or a special character.
- Autoregressive decoding: at each step, predict the next token, add the new token to the context, and continue predicting until generation stops.
- Transformer intuition: use attention to incorporate contextual information, focusing on the words most relevant to the current prediction.
- Fine-tuning continues training after pretraining using task-specific data. Zero-shot learning relies on the model's existing capabilities to perform a task directly. In-context learning provides examples or context in the prompt. RAG augments model responses with external data.

5. Data on the Internet: HTTP, APIs, HTML, and BeautifulSoup

5.1 HTTP Request–Response Model

HTTP (Hypertext Transfer Protocol) is the protocol used by clients and servers to exchange information. The client sends a request; the server processes it and returns a response. The S in HTTPS stands for secure.

Concept	Explanation
Client	The party that initiates a request, such as a browser or Python program.
Server	The party that receives and processes a request and returns a response.
GET	Request data from a specified resource.
POST	Send data to the server.
Query String	The parameter section after ? in a URL, used to request a more specific resource.
Headers	Request/response metadata.
Response Body	The response body, often containing HTML or JSON.

5.2 requests: Sending HTTP Requests

```
import requests



# GET
```

```
res = requests.get("https://example.com")
res.status_code
res.ok
res.text

# POST: send data to the server
post_res = requests.post(
    "https://httpbin.org/post",
    data={"course": "STAT4710J"}
)
```

Status Code	Common Meaning
200	The request succeeded.
400	Bad Request: the request itself has a problem.
404	Not Found: the resource does not exist.
500	Internal Server Error: an internal server error occurred.

- The .text attribute of a Response object usually contains the response text. If the response is JSON, use .json() to obtain the parsed object.
- Use .ok to check whether a request succeeded. The course Notebook notes that 2xx status codes generally indicate success.
- Whether to retry a failed request depends on the error type; sometimes it is reasonable to wait before trying again.

In-Class Practice 5A | HTTP and Web Scraping

A web-scraping script used to run normally but has become noticeably slower without raising errors. What is one likely reason, and how should the request strategy be improved?

Reference Answers & Explanations

Answer: The script may be sending too many requests in a short period, causing the server to throttle or delay responses. Reduce the number of requests, control the request rate, reuse downloaded pages, and prefer an API when available.

5.3 JSON: A Structured Data Format

JSON (JavaScript Object Notation) is a lightweight data-interchange format. A JSON object resembles a Python dictionary, but the two are not the same object or syntax system.

JSON Type	Meaning	Python Equivalent
String	Text enclosed in double quotes.	str

JSON Type	Meaning	Python Equivalent
Number	A numeric value; JSON does not distinguish the semantic categories of Python int and float.	int / float
Boolean	true / false。	True / False
Null	The JSON null value.	None
Array	An ordered list.	list
Object	A collection of key-value pairs; keys must be strings.	dict

```
import json


# Parse a JSON string

data = json.loads(json_text)


# Parse JSON from a requests response

data = res.json()


# Read JSON Lines: each line is a separate JSON object

records = [json.loads(line) for line in open("records.jsonl")]
```

Safety reminder: Do not use eval on raw content from unfamiliar websites. Parse JSON with a JSON parser rather than executing it as a Python expression.

5.4 APIs and Web Scraping

Method	Definition	Advantages	Limitations
API	An interface maintained by the data provider; data is accessed through a specified URL and request parameters.	Data is often cleaner and more current; the provider explicitly manages/permits its use; sometimes it provides access to data not directly displayed on a webpage.	An API may not exist; an API key may be required; usage restrictions may apply.
Web Scraping	Programmatically	May provide access to	HTML can be messy;

Method	Definition	Advantages	Limitations
	download webpage HTML and extract information from the source code.	information displayed on a webpage when no API is available.	parsing and cleaning can be difficult; website structures may change; and a website may not want to be scraped.

5.5 Responsible Scraping

- Prefer using an API whenever possible.
- Plan requests in advance and minimize the number of requests.
- Do not send a large number of requests in a short period. Requests that are too frequent may be throttled, delayed, or blocked by the server.
- Separate page downloading and page parsing into different functions to make debugging easier and avoid duplicate requests.
- Define the schema of the final output DataFrame before beginning scraping and parsing.

5.6 HTML and the DOM Tree

Tag	Purpose
<html>	The entire HTML document.
<head>	Page metadata and related information.
<body>	The main content of the page.
	A logical block.
	An inline logical block.
	A paragraph.
<a>	A hyperlink; a common attribute is href.
<h1>, <h2>, ...	Heading levels.
<img>	An image; common attributes are src and alt.
,	An unordered list and its list items.

The DOM (Document Object Model) represents HTML as a tree. Nodes have parent, child, and descendant relationships; children under the same parent are ordered. Understanding this tree structure helps determine which node to search from.

In-Class Practice 5B | DOM Trees

Given <body> A B <footer>End</footer></body>, how many direct children does body have?

Reference Answers & Explanations

Answer: Two direct children: div and footer. The p elements are children of div, not direct children of body.

5.7 BeautifulSoup: Parsing HTML

Method/Attribute	Return Value / Purpose	Note
.find(tag, attrs=...)	Returns the **first node** that matches the conditions.	Usually returns a single BeautifulSoup Tag; may return None if no match is found.
.find_all(tag, attrs=...)	Returns a list of **all nodes** that match the conditions.	The result is a list that can be iterated over.
.text	Gets the text of a node and its descendants.	It is not an attribute dictionary.
.get('href')	Reads the href attribute from the current Tag.	Must be called on the node that actually contains the attribute.
.children / .descendants	Access child nodes or all descendants.	descendants traverses the entire subtree.

In-Class Practice 5C | find / find_all / get

```
<html>
    <head>
        <title>ZOMBO</title>
    </head>

    <body>
        <h1>Welcome to Zombo.com</h1>
        <div id="greeting">
            <ul>
                <li>This is Zombo.com, welcome!</li>
                <li>This is Zombo.com</li>
                <li>Welcome to Zombo.com</li>
                <li>You can do anything at Zombo.com -- anything at all!</li>
                <li>The only limit is yourself.</li>
            </ul>
        </div>
        <div id="footnotes" class="faded">
            <h3>Footnotes</h3>

            <ol id="footnotes">
                <li>Please consider <a href="paypal.html">donating!</a></li>
                <li>Made in California with <a href="https://reactj.org">React.
            </ol>
        </div>
    </body>
</html>
```

The page shown above contains five `greetings`, each one a list item in an unordered list. The first greeting is `This is Zombo.com, welcome!`, and the last is `The only limit is yourself.`.

Suppose we have parsed the HTML into a `BeautifulSoup` object stored in the variable named `soup`.

Which of the following pieces of code will produce a list of `BeautifulSoup` objects, each one representing a single greeting list item? Select all that apply.

(a) ✔ `soup.find('div').find_all('li')`

(b) `soup.find_all('li', id='greeting')`

(c) ✔ `soup.find('div',id='greeting').find_all('li')`

(d) `soup.find_all('ul/li')`

5.8 Scraping Pipeline: Separate Downloading from Parsing

```
def download_page(page_number):
    url = f"https://example.com/page/{page_number}"
    response = requests.get(url)
    return bs4.BeautifulSoup(response.text)


def parse_item(item_tag):
    title = item_tag.find("h2").text.strip()
    link = item_tag.find("a").get("href")
    return pd.Series({"title": title, "link": link})


def parse_page(soup):
    item_tags = soup.find_all("div", attrs={"class": "item"})
    return pd.DataFrame([parse_item(tag) for tag in item_tags])


def collect_pages(n):
    frames = []
    for i in range(1, n + 1):
        soup = download_page(i)
        frames.append(parse_page(soup))
    return pd.concat(frames, ignore_index=True)
```

Design principle: download_page handles requests; parse_item/parse_page handle parsing. Giving each function one responsibility makes issues easier to locate and reduces unnecessary network requests.

5.9 Nested vs. Flat Data

Nested formats such as HTML, JSON, and XML can represent hierarchical relationships, whereas flat/tabular formats such as CSV are more convenient for direct tabular analysis. If a column contains lists, you may need to expand them or apply one-hot encoding to make filtering and aggregation easier.

6. Visualization: Distributions, Relationships, and Design

6.1 Basic Properties of a Distribution

A distribution describes how the values of a variable occur. A valid representation of a distribution should account for each value exactly once. When represented by counts, the frequencies sum to the total number of observations; when represented by proportions, the proportions sum to 100%.

6.2 Choosing a Plot Based on Variable Type

Plot	Main Use	Variable Type / Relationship	Key Interpretation
Bar Plot	Category frequencies or comparisons between categories and numeric values.	Qualitative / categorical 。	Bar length encodes the value; bar width usually does not encode information.
Histogram	Distribution of a numeric variable.	A single quantitative variable.	Bins numeric values; in a density histogram, bin area represents proportion.
Rug Plot	Shows the position of every observation.	A single quantitative variable.	Each tick corresponds to one data point; suitable for small samples or as a supplement to a histogram.
Box Plot	Compact comparison of numeric distributions.	Quantitative, often grouped by category.	Median, Q1, Q3, IQR, whiskers, and outliers.
Violin Plot	Compares distributions and shows density shape.	Quantitative, often grouped by category.	Width represents density; also displays boxplot information.
Scatter Plot	Shows the relationship between two numeric variables.	Two quantitative variables.	Can reveal linear/nonlinear relationships, changes in spread, and outliers.
Hex Plot	Two-dimensional distributions with large samples.	Two quantitative variables.	Divides the 2D plane into hexagonal bins; color intensity represents frequency/density.
Contour Plot	Two-dimensional density shape.	Two quantitative variables.	Equal-density contours, similar to a two-

Plot	Main Use	Variable Type / Relationship	Key Interpretation
			dimensional density curve.
ECDF	Empirical cumulative distribution.	A single quantitative variable.	Shows the proportion of observations less than or equal to a given value.

In-Class Practice 6A | Choose a Visualization

1. Which plot should be used to compare sales across beverage categories?
2. Which plots can be used to show the distribution of a single continuous variable?
3. Which plots can be used to show the relationship between two continuous variables?

Reference Answers & Explanations

1. Bar plot
2. Histogram, KDE, rug plot, box plot, and similar plots.
3. Scatter plot, 2D density/contour plot, or hex plot.

6.3 Bar Plot vs. Histogram

- A bar plot is usually used for a categorical variable; categories do not need to have a numeric order.
- A histogram is used for a quantitative variable; adjacent bins represent continuous numeric intervals.
- In a bar plot, bar length encodes frequency or value; bar width usually has no meaning.
- In a density histogram, the area of a bin (height × width), rather than its height alone, represents the proportion in that interval.

In-Class Practice 6B | Histograms and Distributions

1. A histogram bin has width 4 and density height 0.05. What proportion of the total sample does this bin represent?
2. A dataset has a pronounced long right tail. Which is usually larger, the mean or the median?
3. If two datasets produce identical histograms, can we conclude that their original observations are identical?

Reference Answers & Explanations

Answer: Area = width × height = $4 \times 0.05 = 0.20$, or 20%.

Answer: Usually mean > median because extreme values in the right tail pull the mean to the right.

Answer: No. Identical binned frequency distributions do not guarantee that the original observations are identical.

6.4 Describing Distributions: Skewness, Tails, Modes, and Outliers

Term	Explanation
Skewed Right / Positive Skew	A long right tail; the mean is usually pulled toward larger values, so mean > median is common.
Skewed Left / Negative Skew	A long left tail; the mean is usually pulled toward

Term	Explanation
	smaller values, so mean < median is common.
Symmetric	The two tails are approximately similar.
Mode	A local or global peak in a distribution.
Unimodal / Bimodal / Multimodal	Represent one peak, two peaks, and multiple peaks, respectively.
Outlier	An observation clearly separated from the main body of the distribution; the definition depends on the rule used.

6.5 Quartiles, IQR, and Box Plots

For a quantitative variable, Q1 is the 25th percentile, Q2 is the 50th percentile (the median), and Q3 is the 75th percentile. The middle 50% of the data lies in [Q1, Q3].

$$\mathbf{IQR = Q3 - Q1}$$

The course uses the common 1.5 × IQR rule to define boxplot whisker boundaries:

$$\mathbf{Lower\ Fence = Q1 - 1.5 \times IQR} \quad | \quad \mathbf{Upper\ Fence = Q3 + 1.5 \times IQR}$$

- The box extends from Q1 to Q3; the line inside the box represents the median.
- The whiskers extend to the most distant data points that remain within the fences (interpret the exact plotting-library implementation according to the course definition).
- Points beyond the fences are marked as outliers.
- Boxplots make it easy to compare multiple distributions but omit some detail.

6.6 Violin Plot

A violin plot resembles a boxplot but adds a smooth density curve on both sides. Its width is meaningful: greater width near a value indicates higher density there. It can reveal multimodal structure while retaining information such as the median, quartiles, and whiskers.

6.7 Scatter, Hex, and Contour Plots

- A scatter plot uses point positions to represent two numeric variables. It can reveal linear or nonlinear relationships and changes in spread.
- Overplotting occurs when many points overlap, making the true number or density of points difficult to see.
- You can reduce point size or add a small amount of random jitter to x/y. Jitter changes the original coordinates, so use it cautiously for display purposes only.
- A hex plot divides the x-y plane into hexagonal bins and uses color to represent density/frequency. It is suitable for large numbers of points.
- A contour plot shows two-dimensional density contours. A jointplot can also display univariate distributions along the margins.

In-Class Practice 6D | Bivariate Relationships and Overplotting

A dataset contains 500,000 records, each with two continuous variables x and y, and the scatter plot has severe overplotting. Propose two alternative visualizations and explain how each encodes information.

Reference Answers & Explanations

Answer: A hex plot divides the plane into hexagonal bins and uses color to represent the frequency/density within each bin. A contour plot uses equal-density lines to represent the two-dimensional distribution. Sampling or transparency can also help, but explain the information loss involved.

6.8 Kernel Density Estimation (KDE)

KDE estimates a probability density function from observed data. The total area under a valid density curve must equal 1. The basic steps are: place a kernel at each data point, normalize each kernel, and sum all kernels.

1. Place a kernel at each observed data point.
2. Normalize each kernel so that the total area contributed by all kernels is scaled by 1/n.
3. Sum the normalized kernels to obtain the KDE curve.

$$f_\alpha(x) = (1/n) \Sigma_i K_\alpha(x, x_i)$$

Concept	Meaning
Kernel	A function used to construct a density; it must be nonnegative and integrate to 1.
Gaussian Kernel	A common kernel centered at observation x_i; in the course illustration, α is also the Gaussian standard deviation.
Boxcar Kernel	Assigns uniform density within a window around an observation and 0 outside the window.
Bandwidth α	Controls KDE smoothness; larger α produces a smoother curve.

Bandwidth trade-off: an α that is too small may produce many local fluctuations; an α that is too large may erase meaningful multimodal structure. For small samples, a rug/dot plot is often more honest than an excessively smoothed KDE.

In-Class Practice 6E | KDE and Bandwidth

1. What should the total area under a KDE curve be?
2. When bandwidth α increases, does the curve usually become smoother or more jagged?
3. With only five observations, would a KDE or a rug/dot plot usually be preferable?

Reference Answers & Explanations

1. 1. A KDE is a probability density estimate.
2. Smoother, although an excessively large bandwidth may hide multimodal structure.
3. A rug/dot plot often shows each observation more directly and avoids excessive smoothing with a small sample.

6.9 Visualization Theory: Marks and Information Channels

A mark represents a data item (for example, a tick on a rug plot or a point on a scatter plot). Encoding maps data to visual positions or attributes. Common channels include x, y, length, area, color, shape, and shading.

Channel / Design Element	Recommendation
X/Y Position	Position is usually easy to compare; keep coordinate scales consistent across comparable plots.
Length	Length is relatively easy to judge and is suitable for bars; do not mislead comparisons with inconsistent baselines.
Angle / Area	Angles and areas are difficult to compare accurately; the course recommends avoiding pie charts, area charts, and word clouds.
Color — Qualitative	Used to distinguish categories; categories have no order, so colors should be easy to tell apart.
Color — Quantitative	Used to represent numeric magnitude; use a sequential palette for low-to-high values and a diverging palette when there is a meaningful center or positive/negative extremes.
Conditioning	Split/facet by category to make important comparisons easier; show small multiples side by side using the same scale.
Context	Provide an informative title, axis labels, legend, reference lines, and a self-explanatory caption.

6.10 Color and Scale

- A perceptually uniform colormap aims to produce consistent perceived changes for equal numeric changes. The course recommends Viridis and notes that Jet/rainbow palettes can be misleading.
- Avoid relying only on red/green combinations to distinguish categories because red-green color-vision differences exist.
- When comparing two lines or multiple datasets, avoid using different scales on the same axis.
- Choose axis ranges that make the data readable. You may zoom in on the main region or use multiple plots for different ranges, but label them clearly.
- Do not hide important information for dramatic effect. The goal is to reveal the data, not to mislead.

6.11 Conditioning, Superposition, and Juxtaposition

Method	Definition	Advantages / Notes
Superposition	Overlay multiple density curves/scatter plots in one figure.	Makes overlaps easy to compare directly; may become cluttered

Method	Definition	Advantages / Notes
		with many categories.
Juxtaposition / Small Multiples	Use multiple side-by-side small plots for different categories, usually with the same scale.	Facilitates cross-category comparisons; scales must remain comparable.
Conditioning	Group or facet by one variable to help inspect the relationship involving another variable.	Can break a complex comparison into more natural contrasts.

6.12 Transformations and Linearization

Sometimes the relationship between the original variables is nonlinear, or extreme values compress the main body of the plot. Applying a log, square-root, or power transformation changes the scale and can help reveal whether the relationship is closer to linear.

- Log and square-root transformations compress larger values, reducing the influence of extreme values on the axis range.
- A power transformation (such as squaring) magnifies differences among larger values.
- A linearized relationship is easier to interpret using slope/intercept and can be useful for subsequent linear models.
- When interpreting a plot, state that the axis has been transformed. Distances on the transformed scale must not be treated as differences on the original scale.

Visualization delivery standard: the title should communicate the takeaway rather than merely describe the plot type. Clear axis labels are required; add a legend, reference lines, important-value markers, and a complete caption when needed.

In-Class Practice 6F | Log Transformation

An income variable is strongly right-skewed, and a few extremely large incomes compress most of the plot toward the left. How does taking the log of the x-axis usually change the plot?

Reference Answers & Explanations

Answer: A log transformation compresses larger values and reduces the influence of extreme values on the axis range, often making a right-skewed distribution easier to inspect. Make clear that the axis is on a log scale.

7. SQL I & II: From Basic Queries to Aggregation and JOINs

7.1 Databases, DBMS, and SQL

A database is an organized collection of data. A DBMS (Database Management System) stores and manages data and provides an interface for accessing it. Compared with a single CSV file, a DBMS can provide reliable storage, query optimization for large datasets, constraint management, and concurrent access control.

- SQL stands for Structured Query Language. It is a specialized language, distinct from Python, used to interact with database systems.
- SQL is declarative: you describe the result you want rather than specifying every computational step as you would in Python.
- The course uses SQLite/DuckDB. Data types and some syntax may differ across SQL systems.

7.2 Tables, Schemas, and Constraints

Term	Meaning
Table / Relation	A data table consisting of rows and columns.
Row / Record / Tuple	A single record.
Column / Attribute / Field	A field/variable.
Schema	The logical structure of a table: column names, data types, and constraints.
PRIMARY KEY	Uniquely identifies each record.
FOREIGN KEY	References the primary key of another table.
NOT NULL	The field cannot be missing.
CHECK	Inserted values must satisfy a specified condition.
DEFAULT	Uses a default value when no value is provided.

```
CREATE TABLE Member (
    member_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER CHECK (age >= 0),
    status TEXT DEFAULT 'active'
);
```

7.3 Basic SELECT Queries

```
SELECT column1, column2
FROM table_name;


SELECT *
FROM table_name;


SELECT column_name AS alias_name
FROM table_name;


SELECT DISTINCT category
FROM table_name;
```

SELECT specifies the output columns, and FROM specifies the data source. * represents all columns. DISTINCT removes duplicates. AS assigns an alias to an output column. SQL statements usually end with a semicolon.

7.4 WHERE: Filtering Rows

```
SELECT *
FROM Orders
WHERE total >= 100;


SELECT order_id, city
FROM Orders
WHERE total >= 100 AND city IN ('East', 'North');


SELECT *
FROM Orders
WHERE status IS NOT NULL;
```

Category	Operator / Syntax
Comparison	=、>、<、>=、<=、!=
Logic	AND、OR、NOT
Set Membership	IN (...)
Text Pattern	LIKE

Category	Operator / Syntax
NULL	IS NULL、IS NOT NULL

NULL pitfall: Do not use = NULL or != NULL to test for missing values; use IS NULL / IS NOT NULL. SQL strings usually use single quotes, for example 'East'.

7.5 ORDER BY, LIMIT, OFFSET, and Sampling

```
SELECT *
FROM Orders
ORDER BY total DESC, order_date ASC
LIMIT 10 OFFSET 5;


-- Random sampling example (syntax may vary by SQL dialect)
SELECT *
FROM Orders
ORDER BY RANDOM()
LIMIT 5;
```

- ORDER BY defaults to ASC (ascending); DESC specifies descending order.
- LIMIT n returns only the first n rows; OFFSET k skips the first k rows.
- Randomly sorting and then applying LIMIT can create a random sample; the specific random function varies by SQL dialect.

7.6 Aggregation and GROUP BY

Aggregate Function	Purpose
COUNT(*)	Counts the total number of rows in a group, including rows where a field is NULL.
COUNT(column)	Counts the number of non-NULL values in the column.
AVG(column)	Average.
SUM(column)	Sum.
MIN(column) / MAX(column)	Minimum / maximum.

```
SELECT department, AVG(score) AS avg_score
FROM Results
```

```
GROUP BY department;
```

GROUP BY partitions rows with the same key values into groups, then aggregate functions reduce each group to a summary. Non-aggregated columns in SELECT should generally correspond to the grouping columns in GROUP BY.

7.7 WHERE vs. HAVING

WHERE	HAVING
Filters original rows (row-level filtering).	Filters groups after grouping (group-level filtering).
Executed before GROUP BY.	Executed after GROUP BY.
Generally not used directly to filter aggregate statistics for groups.	Often used with aggregate conditions such as COUNT, SUM, AVG, MIN, and MAX.

```
SELECT department, COUNT(*) AS n_records
FROM Results
WHERE status = 'complete'       -- filter rows first
GROUP BY department
HAVING COUNT(*) >= 5            -- filter groups afterward
ORDER BY n_records DESC;
```

In-Class Practice 7A | WHERE and HAVING

Using the Orders table, analyze only orders with status='paid', group them by region, and retain only regions with at least three paid orders. Write the row-level and group-level filtering conditions.

Reference Answers & Explanations

Answer: WHERE status = 'paid'; GROUP BY region; HAVING COUNT(*) >= 3. WHERE filters rows first, while HAVING filters groups after grouping.

7.8 SQL Logical Execution Order

4. FROM: retrieve the source tables.
5. JOIN: combine relations according to the join condition.
6. WHERE: filter rows.
7. GROUP BY: create groups.
8. HAVING: filter groups.
9. SELECT: compute output expressions and aggregated columns.
10. DISTINCT: remove duplicates.
11. ORDER BY: sort the results.
12. LIMIT / OFFSET: restrict the output rows.

Important: SQL is not executed in the top-to-bottom statement order used in Python. In particular, remember that WHERE is applied before GROUP BY and HAVING is applied after GROUP BY.

7.9 LIKE, CAST, and CASE

```
-- LIKE: simple text pattern matching
SELECT title
FROM Movie
WHERE title LIKE 'The%';


-- CAST: convert a data type
SELECT CAST(runtime AS INTEGER) AS runtime_int
FROM Movie;


-- CASE: create a new column based on conditions
SELECT title,
       CASE
          WHEN year < 1980 THEN 'classic'
          WHEN year < 2010 THEN 'modern'
          ELSE 'recent'
       END AS era
FROM Movie;
```

- LIKE is used for simple text matching. Common wildcards: % represents a string of any length, and _ represents a single character (specific behavior depends on the SQL dialect used in the course).
- CAST(col AS TYPE) creates a column with a converted data type, similar to Pandas astype.
- CASE WHEN ... THEN ... WHEN ... THEN ... ELSE ... END is similar to Python if/elif/else.

In-Class Practice 7B | CASE and NULL

1. The Cats table has an age column. Use CASE to create Nickname: 'Kitten' when age<=1, 'Wise One' when age>=15, and 'Catto' otherwise.

2. What condition should be used to select records where name is not NULL?

Reference Answers & Explanations

Answer: CASE WHEN age <= 1 THEN 'Kitten' WHEN age >= 15 THEN 'Wise One' ELSE 'Catto' END AS Nickname.

Answer: WHERE name IS NOT NULL; do not write name != NULL.

7.10 JOIN: Combining Multiple Tables

Join Type	Result
INNER JOIN	Keeps only rows with matching keys on both sides; unmatched rows are dropped.
LEFT JOIN	Keeps all rows from the left table and matching rows

Join Type	Result
	from the right; unmatched values are filled with NULL.
RIGHT JOIN	Keeps all rows from the right table and matching rows from the left; unmatched values are filled with NULL.
FULL OUTER JOIN	Keeps all rows from both tables; fields on the unmatched side are filled with NULL.
CROSS JOIN	The Cartesian product of all rows from both tables; no matching key is required.

```
SELECT C.customer_name, O.total
FROM Customers AS C
INNER JOIN Orders AS O
    ON C.customer_id = O.customer_id;
```

- JOIN appears in the FROM clause; ON specifies the matching condition.
- Table aliases (such as AS C and AS O) make SQL more concise and avoid ambiguity when columns share the same name.
- INNER JOIN is the most common matching join used in the course. In Pandas, pd.merge defaults to how='inner'.
- After a JOIN, check the granularity of the output: one customer may have multiple orders, so one customer row may expand into multiple rows.

In-Class Practice 7C | JOIN and HAVING

There are two tables: Quests(location, reward) and Travel(location, travel_cost). Keep only locations that appear in both tables, output location and average reward, and require the total reward for that location to exceed travel_cost. Write the query structure.

Reference Answers & Explanations

Answer: SELECT q.location, AVG(q.reward) AS avg_reward FROM Quests AS q INNER JOIN Travel AS t ON q.location=t.location GROUP BY q.location, t.travel_cost HAVING SUM(q.reward)>t.travel_cost.

Link, a heroic knight, has turned to data science to help him optimize his adventures. He creates the SQL table quests to record the quests he has been offered on his journeys. The first few rows of quests are shown below. The table contains three columns:
`name`: the name of the person offering the quest
`location`: the location of the quest
`reward`: the monetary reward for completing the quest

name	location	reward
Hestu	Woodlands	10
Impa	Village	50
Sidon	Lake	20
Tulin	Mountain	5
Paya	Village	45
NULL	Underground	100

He creates a second table, `travel`, to store the travel cost associated with journeying to each location. The first few rows of travel are shown below.

location	travel_cost
Village	3
Lake	25
Woodlands	10
Mountain	50

Fill in the blanks below to write a SQL query that returns a table with two columns:
location: a location with one or more quests
avg_reward: the average reward of all quests in that location

Only include locations where the total reward of all quests in the location is **greater than** the travel cost of going to that location. If a location is not present in one or both of Link's tables, this location should be **excluded**.

```
SELECT q.location, ________A________ AS avg_reward
FROM quests AS q ________B________ travel AS t
ON q.location = t.location
_________________C_________________
_________________D_________________;
```

Solution: A: AVG(reward)
B: INNER JOIN (or JOIN, which defaults to an inner join)
C: GROUP BY q.location or GROUP BY t.location
D: HAVING SUM(reward) > travel_cost □

7.11 Conceptual Mapping Between SQL and Pandas

SQL	Pandas Analogy
WHERE	Boolean filtering df[mask]
GROUP BY + aggregate	df.groupby(...).agg(...)
HAVING	The group-level filtering intuition of groupby.filter(...)
JOIN	pd.merge(...)
ORDER BY	sort_values(...)
LIMIT	head(n)

7.12 Comprehensive Query Template

```
SELECT
    C.region,
    COUNT(*) AS n_orders,
    AVG(O.total) AS avg_total,
    SUM(O.total) AS sum_total
FROM Customers AS C
INNER JOIN Orders AS O
    ON C.customer_id = O.customer_id
WHERE O.status = 'complete'
GROUP BY C.region
HAVING SUM(O.total) > 1000
ORDER BY avg_total DESC
LIMIT 10;
```

When reading this type of question, first identify: which columns to output (SELECT), where the data comes from (FROM/JOIN), which original rows to retain (WHERE), how to group them (GROUP BY), which groups satisfy the condition (HAVING), and how to sort and limit the results (ORDER BY/LIMIT).

In-Class Practice 7D | Integrated SQL Queries

1. In the Cats table, first filter for age>1, then find owner_id values with more than one qualifying cat.
2. Find the owner_id with the largest number of cats.
3. Return all cat records whose breed ranks in the top two by frequency.

Reference Answers & Explanations

1. SELECT owner_id FROM cats WHERE age>1 GROUP BY owner_id HAVING COUNT(*)>1 。
2. SELECT owner_id FROM cats GROUP BY owner_id ORDER BY COUNT(*) DESC LIMIT 1 。
3. SELECT * FROM cats WHERE breed IN (SELECT breed FROM cats GROUP BY breed ORDER BY COUNT(*) DESC LIMIT 2)。

8. Machine Learning Supplement

8.1 Supervised Learning and Multiple Linear Regression

Supervised learning uses labeled data $(x^{(i)}, y^{(i)})$ to learn a mapping from the feature space to the target space. Multiple Linear Regression predicts a numeric target using a linear combination of multiple features.

$$\hat{y}^{(i)} = \theta_0 + \theta_1 x_1^{(i)} + \ldots + \theta_p x_p^{(i)}$$

- Features: input variables x; target: the y value to be predicted.
- θ_0 is the intercept/bias term; $\theta_1 \ldots \theta_p$ are the feature coefficients.
- The design matrix X has n rows and p+1 columns, including the intercept column.
- Predictions for all observations can be written as $\hat{y} = X\theta$; residuals are $r = y - X\theta$.

8.2 MSE and Ordinary Least Squares (OLS)

$$MSE(\theta) = (1/n) \Sigma_i (y^{(i)} - \hat{y}^{(i)})^2$$

OLS chooses parameters θ to minimize the sum of squared residuals on the training data. If X^TX is invertible, there is a unique closed-form solution:

$$\hat{\theta} = (X^TX)^{-1} X^Ty$$

Condition for a unique solution: If columns of the design matrix are linearly dependent (for example, all dummy variables are retained along with an intercept, causing perfect multicollinearity), X^TX may be non-invertible and the OLS parameters may not be unique.

In-Class Practice 8A | Design Matrices and OLS

A model uses three nominal features: product with 4 categories, shipping method with 3 categories, and region with 2 categories. It applies one-hot encoding(drop_first=True) and then adds an intercept. How many columns will the design matrix have?

Reference Answers & Explanations

Answer: (4−1)+(3−1)+(2−1)+1 = 8 columns. One baseline category is dropped for each categorical variable, and the intercept is added afterward. A unique OLS solution also requires the design matrix to have full column rank.

8.3 Model Evaluation

Metric	Formula / Explanation

Metric	Formula / Explanation
Training MSE	$(1/n)\,\Sigma(y_i - \hat{y}_i)^2$; mean squared prediction error.
RMSE	$\sqrt{}$MSE; measured in the same units as the target.
R^2	1 – SS_res / SS_tot; values close to 1 indicate that the model explains more variation in y, while values close to 0 indicate limited improvement over predicting the mean.

8.4 Holdout and K-Fold Cross-Validation

- Holdout: randomly split the data into a training set and a validation set. Use the training set to fit model parameters and the validation set to evaluate the model or hyperparameters.
- Holdout is simple and fast, but its error estimate depends on one random split and may have high variance. For a small dataset, reserving a large validation set also reduces the amount of training data.
- K-fold CV: divide the data into K approximately equal folds. At each iteration, use one fold for validation and the remaining K–1 folds for training. Repeat K times and average the validation scores.
- If you compare C candidate hyperparameter settings using K-fold CV, you generally calculate C × K validation scores.

In-Class Practice 8B | Cross-Validation

How many validation scores are required to compare four candidate λ values using 5-fold cross-validation?

Reference Answers & Explanations

Answer: 4×5=20 validation scores. Each candidate λ must be evaluated once on each of the five folds.

8.5 Regularization: Controlling Model Complexity

Regularization adds a parameter-penalty term to the loss function, with $\lambda \geq 0$ controlling the penalty strength. A general objective has the form:

$$\mathbf{MSE(\theta) + \lambda \cdot Penalty(\theta)}$$

Method	**Penalty**	Main Effect
Ridge / L2	$\Sigma_j\, \theta_j^2$	Shrinks coefficients toward 0, usually without making them exactly 0; can reduce variance and help with correlated features.
LASSO / L1	$\Sigma_j\, \lvert\theta_j\rvert$	Can shrink some coefficients to exactly 0, producing a sparse model and performing feature selection.

- The larger λ is, the stronger the regularization and the more strongly the coefficients are shrunk.
- Ridge usually retains all features; LASSO can automatically eliminate some features.
- λ is usually selected using cross-validation.

In-Class Practice 8C | L1/L2 and Overfitting

1. If you want the model to shrink some coefficients to exactly zero and perform feature selection, should you prefer L1 or L2?

2. Training error decreases rapidly while validation error continues to rise. What type of adjustment should you consider?

Reference Answers & Explanations

1. L1 / LASSO 。

2. This is a typical sign of overfitting. Consider increasing regularization (increasing λ) or reducing model complexity.

8.6 Bias–Variance and Overfitting

Concept	Intuition	Common Response
Bias	The model is too simple to capture the true relationship systematically; often associated with underfitting.	Increase model complexity or add useful features appropriately.
Variance	The model is overly sensitive to changes in the training sample; often associated with overfitting.	Reduce complexity, add data, or use regularization.
Irreducible Noise	Randomness inherent in the data that cannot be eliminated simply by changing the model.	Recognize its presence; not all error should be attributed to the model.

A common intuition for decomposing model risk is Risk ≈ Bias² + Variance + Irreducible Noise. If training error keeps decreasing while validation error increases, this usually indicates overfitting; consider increasing regularization or reducing model complexity.

In-Class Practice 8D | Bias–Variance

A high-complexity model produces very different fitted curves on different training sets. Is this primarily associated with high bias or high variance?

Does $\mathrm{Var}(\hat{Y}(x))$ correspond to bias, variance, or total risk?

Reference Answers & Explanations

Answer: High variance; $\mathrm{Var}(\hat{Y}(x))$ represents model variance. If the model is too complex, reduce its complexity, increase the training data, or use regularization.

8.7 One-Hot Encoding and Design Matrices

Nominal categories should not be arbitrarily encoded as 1, 2, and 3 in alphabetical order, because this introduces nonexistent magnitude/distance relationships. One-hot encoding creates an indicator variable for each category. With k categories, full one-hot encoding usually produces k columns; drop='first' removes one column.

In-Class Practice 8E | Nominal Encoding

A color variable has three categories: blue, green, and red. Why is it inappropriate to encode them directly as 1, 2, and 3? How many columns are produced by one-hot encoding(drop_first=True)?

Reference Answers & Explanations (TA Version)

Answer: Color is nominal and has no natural order; encoding it as 1, 2, and 3 introduces nonexistent magnitude/distance relationships. drop_first=True produces 3−1=2 columns.